Name:- Aaryan N. Rodrane Roll No:- 5004 Div:- 5
batch:- A1 SAP ID:- 60018250070 branch:- AI&DS

Experiment No 4

* Title:- Sequences and dictionary

* Aim:- To understand and utilize Python collections (lists, tuples, sets and dictionaries) and explore their applications in data science and statistics.

* Theory:-

1) What is a list in Python? How is it different from an array in other programming languages?

⇒ A list is an ordered, mutable collection in Python that can store elements of different data types. Arrays in languages like C or Java require all elements to be of the same type and have a fixed size, whereas Python lists resize dynamically and accept mixed types.

Feature	Python list	Array (C / Java)
data types	Multiple types allowed	single type only
size	Dynamic	Fixed at declaration
Memory layout	Non contiguous	contiguous

Sundaram
FOR EDUCATIONAL USE

code:
```
data = [85, 90.5, "A", True]
print(data)
```

output: [85, 90.5, 'A', True]

2) Explain the concept of list Mutability with an example.

→ A list is mutable meaning its contents can be modified after creation. Elements can be updated, added or removed without replacing the entire list object.

code:
```
marks = [70, 80, 90]
marks[0] = 95
print(marks)
```

output: [95, 80, 90]

3) Differentiate between append() and extend() methods.

⇒

Method	Behaviour	Argument type
append()	adds a single element to the end	any object
extend()	unpacks and adds each element of an iterable	Iterable (list, tuple, etc.

Sundaram
FOR EDUCATIONAL USE

code:
```
a = [1, 2, 3]
a.append([4, 5])
print(a)

b = [1, 2, 3]
b.extend([4, 5])
print(b)
```

output:
```
[1, 2, 3, [4, 5]]
[1, 2, 3, 4, 5]
```

4) what is the difference between list.sort() and sorted(list)?

=>

feature	list.sort()	sorted(list)
Modifies original list	Yes (in place)	No
Return value	None	A new sorted list
works on	Lists only	Any iterable

code:
```
a = [3, 1, 2]
b = sorted(a)
print("sorted():", a, b)
a.sort()
print("list.sort():", a)
```

output:
```
sorted(): [3, 1, 2] [1, 2, 3]
list.sort(): [1, 2, 3]
```

Sundaram
FOR EDUCATIONAL USE

5) What is a tuple in Python? How is it different from list?

=> A tuple is an ordered, immutable collection defined using parentheses. Its elements cannot be changed once assigned.

Feature	Tuple	List
Mutability	immutable	Mutable
Syntax	()	[]
Performance	Faster	Slower
Use case	fixed/constant data	Dynamic data

Code :-
```
cords = (10.5, 20.3)
print (cords[0])
```

Output :- 10.5

6) Why are tuples called immutable? What are the advantages of immutability?

=> Tuples are immutable because their elements cannot be modified, added or deleted after creation. Any attempt to do so raises a TypeError.

The advantage of tuple is that they are safe for storing data that should not change, they can be used as dictionary keys (since they are hashable), and they iterate faster than

Sundaram
FOR EDUCATIONAL USE

lists due to lower memory overhead

code :-
```
t = (1, 2, 3)
t[0] = 99
```

output :- TypeError: 'tuple' object does not support item assignment

7) Can a tuple contain mutable objects like lists? Explain with an example.

=> Yes, the tuple itself remains immutable (its references cannot be reassigned), but a mutable object stored inside it can still be modified through its own methods

code :-
```
t = (1, [2, 3], 4)
t[1].append(99)
print(t)
```

output :- (1, [2, 3, 99], 4)

8) What is a set in Python and what are its main properties?

=> A set is an unordered collection of unique, hashable elements. It is defined using curly braces or the set() constructor. Sets support mathematical operations such as union, intersection and difference.

Sundaram
FOR EDUCATIONAL USE

Main properties :-

- unordered (no indexing)
- duplicates not allowed
- elements must be immutable

Code :-
```
s = {10, 20, 30, 20, 10}
print(s)
print(type(s))
```

Output :-
```
{10, 20, 30}
<class 'set'>
```

q) How are sets different from lists and tuples?

⇒

Feature	set	List	Tuple
order	unordered	ordered	ordered
duplicates	not allowed	allowed	allowed
Mutability	Mutable	Mutable	Immutable
Indexing	Not supported	supported	supported

Code :-
```
lst = [1, 2, 2, 3]
tpl = (1, 2, 2, 3)
st = {1, 2, 2, 3}
print(lst, tpl, st)
```

output :- [1, 2, 2, 3] (1, 2, 2, 3) {1, 2, 3}

Sundaram
FOR EDUCATIONAL USE

10) Why are duplicates not allowed in a set?

⇒ sets use a hash table internally. Each element is stored at a position determined by its hash value. When a element with an identical hash (ie, or equal value) is inserted, it replaces the existing entry rather than creating a new one. This mechanism enforces uniqueness automatically.

code:-
```
s = set([4,4,5,5,6])
print(s)
```

output: {4,5,6}

11) What is the difference between remove() and discard() methods in sets?

⇒

method	when element is not found
remove()	Raises KeyError
discard()	Does nothing, no error raised

code:-
```
s={1,2,3}
s.discard(99)
print(s)
s.remove(99)
```

output:-
```
{1,2,3}
KeyError: 99
```

Sundaram FOR EDUCATIONAL USE

(2) what is a dictionary in Python? How is it different from a list?

⇒ A dictionary is an unordered collection of key value pairs. Elements are accessed using keys rather than numeric indices.

Feature	Dictionary	List
Access Method	By Key	By Index
Element structure	Key value pairs	single values
Key requirement	Must be immutable	Not applicable

code :- student = {"name": "Raj", "Marks": 88}
print (student ["name"])

output :- Raj

(3) Explain the concept of Key-value pairs in dictionaries.

⇒ Every entry in a dictionary has two parts: a key and a value. The key acts as a unique identifier and the value holds the associated data. Keys must be unique within a dictionary, but values can repeat.

Sundaram
FOR EDUCATIONAL USE

code :- course = {
"subject" : "Python",
"code" : "DJS23SLPC303",
"credits" : 2
}
print(course["subject"], "-->", course["code"])

output :- Python --> DJS23SLPC303

(a) Why must dictionary keys be immutable (like strings, numbers, tuples)?

=> Python uses a hash table to store dictionary keys. Hashing requires the value of an object to remain constant. Mutable objects like lists have no stable hash, so attempting to use them as keys raises a TypeError.

code :- d = {}
d[(1,2)] = "valid"
print(d)
d[[1,2]] = "invalid"

output :- {(1,2) : 'valid'}
TypeError : unhashable type : 'list'

Sundaram
FOR EDUCATIONAL USE

(5) Differentiate between get() and direct key access (eg. d['key']).

⟹

Feature	get()	d['key']
Key not found	Returns none or a specified default	Raises KeyError
safety	Safe for unknown Keys	Risky without prior check
Default value	can be specified	Not applicable

code:-
```
d={"a":100}
print(d.get("b",0))
print(d["b"])
```

output:-
```
0
KeyError: 'b'
```

*Conclusion:-

This experiment covers the four built in collection types in Python, namely lists, tuples, sets, and dictionaries, each differing in mutability, ordering, and element constraints. Lists and tuples handle ordered sequence while sets enforce uniqueness through hashing and dictionaries provide fast key based data access. Selecting the appropriate collection type based on these properties is essential for writing correct and efficient data science programs.

Sundaram
FOR EDUCATIONAL USE